

School: Glory Academy

Class: level 3 NIT

Module name: GENCP 401 C PROGRAMMING

Competency: APPLY C PROGRAMMING

Learning Hours: 100

TRAINER: YANKURIJE Gad

Elements of Competency and Performance Criteria	
Elements of competency	Performance criteria
1. Apply Computer programming Languages	1.1. Programming languages are properly identified considering their types and application
	1.2. Algorithm is conveniently developed considering the problem to be solved
	1.3. Flowchart is accurately developed based on the logic of question
2. Write C Programming codes	2.1. C programming building blocks are properly described based on programming standards
	2.2. C program structure is property described according to programming standards
	2.3. Condition statements are effectively applied according to the logic of the problem to be solved.
	2.4. Loops are effectively applied according to the logic of the problem to be solved.
	2.5. Functions are effectively applied according to the logic of the problem to be solved.
	2.6. Arrays are effectively applied according to the logic of the problem to be solved.
	2.7. C problems are correctly solved by using C programming techniques

3. Perform Program testing Debugging	3.1. Different program errors are carefully identified according to the programming standards
	3.2. The program is systematically compiled in accordance with the program instructions.
	3.3. The program is tested and errors are completely corrected based on the program and programming instructions.

Learning outcome ONE: Apply Computer programming Languages

1.1. Programming languages are properly identified considering their types and application

Identification of programming Language

What is programming? Is the process of writing instructions that a program needs to execute. These instructions are written in programming languages, which are designed to communicate with the computers.

What is computer program? It is a sequence or set of instructions in programming language for a computer to execute.

What is programming language? It is a computer language programmers use to develop software programs, scripts, or other sets of instructions for computers to execute.

✓ Types of programming languages

Types of programming languages

While you'll find dozens of ways to classify various programming languages, they generally fall into five major categories. Keep in mind that some languages may fall under more than one type:

1. Procedural programming languages

A procedural language follows a sequence of statements or commands in order to achieve a desired output. Each series of steps is called a procedure, and a program written in one of these languages will have one or more procedures within it. Common examples of procedural languages include:

- C and C++
- Java

- Pascal
- BASIC

2. Functional programming languages

Rather than focusing on the execution of statements, functional languages focus on the output of mathematical functions and evaluations. Each function—a reusable module of code—performs a specific task and returns a result. The result will vary depending on what data you input into the function. Some popular functional programming languages include:

- Scala
- Erlang
- Haskell
- Elixir
- F#

3. Object-oriented programming languages

This type of language treats a program as a group of objects composed of data and program elements, known as attributes and methods. Objects can be reused within a program or in other programs. This makes it a popular language type for complex programs, as code is easier to reuse and scale. Some common object-oriented programming (OOP) languages include:

- Java
- Python
- PHP
- C++
- Ruby

4. Scripting languages

Programmers use scripting languages to automate repetitive tasks, manage dynamic web content, or support processes in larger applications. Some common scripting languages include:

- PHP
- Ruby
- Python
- bash

- Perl
- Node.js

5. Logic programming languages

Instead of telling a computer what to do, a logic programming language expresses a series of facts and rules to instruct the computer on how to make decisions. Some examples of logic languages include:

- Prolog
- Absys
- Datalog
- Alma-0

Other ways to classify programming languages

You'll find many more ways to categorize languages beyond the five listed above. Let's take a closer look at there other ways you can think about programming languages:

Front-end vs. back-end languages

Front-end languages are primarily concerned with the ‘user’ aspect of the software. The front end deals with all of the text, colors, buttons, images, and navigation that the user will face when navigating your website or application. Anyone with a background in graphic design or art may be more inspired to begin learning one of the front-end languages.

Some examples of front-end programming languages include:

- HTML
- CSS
- JavaScript
- React

Back-end languages deal with storage and manipulation of the server side of software. This is the part of the software that the user does not directly come into contact with but supports their experience behind the scenes. This includes data architecture, scripting, and communication between applications and underlying databases.

Anyone with experience in mathematics or engineering may find more interest in back-end development.

Some examples of back-end programming languages include:

- JavaScript
- PHP
- Java
- Python
- Ruby
- C#

Role of programming languages in electronics and telecommunication

Programming is a skill or tool that is used for commanding or instructing computers or hardware devices to do a specific task most especially those that are repetitive and need precision because a human can't, or those tasks that help to make life easier for us.

Where is programming needed in EEE?

Not all aspect of electrical and electronics engineering requires programming but the areas where they are needed most includes:

1. Embedded systems

Embedded systems are referred to as small computer (or computing device other than the normal computers you know of) that is designed to perform a specific task. e.g. a calculator.



2. Software design

Many software's are needed to control devices or to make simulations of real-world problems.

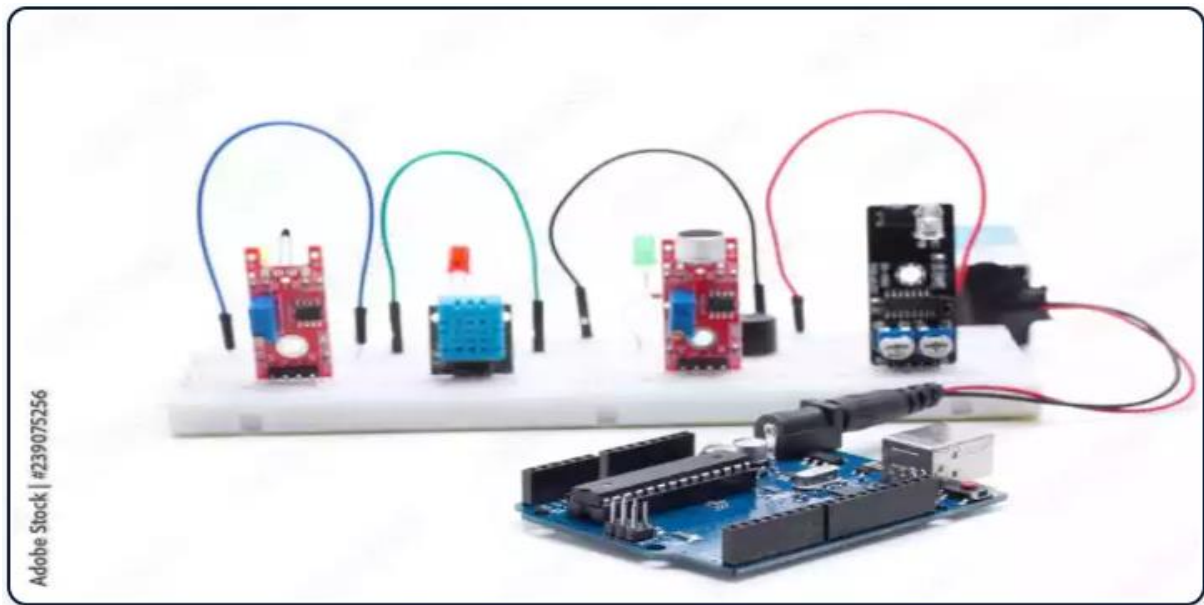
For example, a software like Everycircuit or Multisim that allows you to prototype circuits and know the value of voltages across and currents through each component in the circuit even before designing the circuit which is a way to making predictions. Hence making things much faster, better, and quite efficient.



3. IoT or the internet of things

This is a trending technology and it is growing rapidly because it utilizes microcontrollers to make smart devices or systems.

To know how important IoT is, have you ever desired to control your devices or appliances at home e.g. air conditioner, or detect gas leakage while you are not even at home, if yes then IoT is the technology that can help you do that.



✓ Types of C programming IDE

The Integrated Development Environment, or IDE for short, is a piece of software or an application that programmers use to write code. It makes programming easier for programmers by giving all of the necessary tools for software development.

An IDE is made up of three main components:

- a source code editor,
- a build automation tool (compiler), and
- a debugger.

Programmers use the **source code editor** to create code, while the **build automation tool** is used to compile the code, and the **debugger** is used to test or debug the program in order to address any errors in the code. These IDEs also include functionality such as object and data modeling, as well as unit testing.

Different Types of IDE

There are many ways developers work to product the different types of code they produce, which means there is also a variety of IDEs to use. Some are designed to work with one specific language, while others are cloud-based IDEs, IDEs customized for the production of mobile applications or for HTML, and also IDEs that are meant specifically for Apple or Microsoft development.

Multi-Language IDE

The multi-language IDEs – like Eclipse, Aptana, Komodo, NetBeans, and Geany – support multiple programming languages.

- **Eclipse:** Supports C, C++, Perl, Python, Ruby, PHP, Java and others. It is a free and open source editor for many development frameworks. Although it began as a Java development environment, it has expanded through plugins. This IDE is managed and directed by the Eclipse.org Consortium.
- **NetBeans:** Supports Java, PHP, JavaScript, C, C++, Python, Ruby, and more. It is also free and open source. Modules provide all functions of the IDE. Developers can add support for other programming languages by installing additional modules.
- **Komodo IDE:** Supports Perl, PHP, Python, Tcl, JavaScript, Ruby, and more. This is an enterprise-level tool with a higher price point.
- **Aptana:** Supports HTML, JavaScript, CSS, AJAX and others through plugins. It is a popular choice for programmers who do web app development.
- **Geany:** Supports C, PHP, Java, HTML, Perl, Python, Pascal and many more. This is a very customizable environment with a big set of plugins.

IDE for Mobile Development Processes

Specifically for mobile development, there are IDEs that include PhoneGap and Titanium Mobile from Appcelerator.

A lot of IDEs, particularly multi-language IDEs, have mobile-development plugins. Eclipse, for example, has this functionality.

HTML IDE

IDEs for developing HTML applications are some of the most popular IDEs around. For example, DreamWeaver, HomeSite, and FrontPage automate numerous tasks involved in the process of website development.

Cloud-Based IDE

Starting to become more mainstream, cloud-based IDEs are something to keep an eye on. The capabilities of such web-based IDEs are growing rapidly; for that reason, most major vendors will likely need to offer one if they want to stay competitive in their markets. Cloud IDEs are important because they give programmers access to their code from anywhere.

For example, Nitrous is a cloud-based development environment platform that supports Ruby, Python, Node.js and more. Cloud9 IDE supports more than 40 languages, including PHP, Ruby, Python, JavaScript with Node.js, and Go. Heroku is a cloud-based development platform as a service (PaaS), supporting several programming languages.

IDE That Are Specific to Apple or Microsoft

The following IDEs accommodate programmers who are working in Microsoft or Apple environments:

- **Visual Studio:** Supports VB.NET, Visual C++, C#, F# and more. Visual Studio is Microsoft's IDE, designed to create apps for the Microsoft platform.
- **MonoDevelop:** Supports Visual Basic, C/C++, C#, and additional .NET languages.
- **Xcode:** Supports Swift and Objective-C languages, as well as Cocoa and Cocoa Touch APIs. This IDE is solely for creating iOS and Mac applications. It includes a GUI builder and an iPhone/iPad simulator.
- **Espresso:** Supports XML, HTML, CSS, PHP, and JavaScript. Espresso is a tool specifically for Mac web programmers.
- **Coda:** Supports PHP, CSS, HTML, JavaScript, AppleScript, and Cocoa API. This IDE is highlighted as "one-window development" for Mac users.

Setting up C programming environment

If you want to set up your environment for C programming language, you need the following two software tools available on your computer, (a) Text Editor and (b) The C Compiler.

Text Editor

This will be used to type your program. Examples of few a editors include Windows Notepad, OS Edit command, Brief, Epsilon, EMACS, and vim or vi.

The name and version of text editors can vary on different operating systems. For example, Notepad will be used on Windows, and vim or vi can be used on windows as well as on Linux or UNIX.

The C Compiler

The source code written in source file is the human readable source for your program. It needs to be "compiled", into machine language so that your CPU can actually execute the program as per the instructions given.

The compiler compiles the source codes into final executable programs. The most frequently used and free available compiler is the GNU C/C++ compiler, otherwise you can have compilers either from HP or Solaris if you have the respective operating systems.

1.2.Algorithm is conveniently developed considering the problem to be solved

- **Development of an algorithm**

What is algorithm?

An algorithm is a procedure or step-by-step instruction for solving a problem. They form the foundation of writing a program.

For writing any programs, the following has to be known:

- Input
- Tasks to be preformed
- Output expected

For any task, the instructions given to a friend is different from the instructions given to a computer. Let's look at the difference between them and know how to give instructions to a computer.

Instruction to a Friend:

Step 1: Go to the College Board.

Step 2: Search for my name on the board.

Step 3: Check the rank against my name.

Instructions to the Computer:

Step 1: Go to the College Board.

Step 2: Read the first name.

Step 3: If this is my name, see rank and come back.

Step 4: Read the next name.

Step 5: Repeat the above two steps until you find my name.

Breaking down the steps into smaller steps is called algorithm.

Example:

Considering the above example, let's break down the task into steps:

- Input – Name
- Tasks to be performed – Search for my name
- Output expected – My rank

It is a good practice to write down the algorithm first before attempting at writing a program. Each step of the algorithm will get converted to a line or a set of lines in the programming language.

✓ Types of algorithms

Type of algorithms used in C language.

1. Sorting algorithms

C provides a rich set of data types and operators that can be used to implement various sorting algorithms such as bubble sort, insertion sort and quick sort.

These algorithms are useful in many applications because they can be used to sort data of different sizes and types.

There are different sorting algorithms.

they are:-

(i) Bubble sort: An uncomplicated sorting algorithm that compares nearby components repeatedly and switches them out if they are out of order.

The Algorithm for Bubble sort is:-

1. Start with an unsorted list of elements.
2. Compare the first two elements in the list. If the first element is larger than the second element, swap them.
3. Move on to the next pair of elements and repeat step 2 until the end of the list is reached.
4. For each item on the list, repeat steps 2 and 3 once more. that is referred to as passes.
5. Repeat steps 2-4 for the entire list. As you repeat the passes, elements will "bubble up" to their correct position in the sorted list.
6. Once a pass is completed and no swaps are made, the list is sorted, and the algorithm can stop.
7. The final sorted list is returned.

(ii) Insertion sort: a method of sorting that creates a sorted list one individual element at a time by placing each one in the appropriate spot.

The Algorithm for Insertion sort is:-

1. Initialize an empty sorted list and an unsorted list of the elements to be sorted.
2. The first member from the unsorted list should be taken and placed in the appropriate position in the sorted list.
3. Repeat step 2 for each subsequent element in the unsorted list.
4. Compare the current element with the elements in the sorted list, starting with the element immediately to the left.
5. Swap the two elements if the current element is smaller than the element to its left.
6. If the current element is larger than the element to its left, insert it at its correct position in the sorted list.

7. Repeat steps 4-6 for each subsequent element in the unsorted list.
8. Once all elements have been processed, the sorted list will contain all elements in the correct order.
9. The sorting process is complete.

(iii) Selection sort: a method of sorting that consistently starts the sorted listing with the smallest detail from the unordered listing.

The Algorithm for Selection sort is:-

1. Begin by setting the primary element of list as the min element.
2. Repeat through the remaining items in the list, comparing each one to the current minimum.
3. Set a new minimum if an element is found to be smaller than the existing one.
4. Change the current minimum to the first element of the list whenever it reaches its conclusion.
5. For the remaining unsorted portion of the listing, repeat steps 2-4, but begin with the second item on the list (as the first element is already sorted).
6. Continue sorting the list in this manner until it is all sorted.

(iv) Quick sort: A divide-and-conquer sorting algorithm that chooses a pivot element and splits the list into sublists depending on whether the elements are fewer than or more than the pivot. After that, the sublists are sorted repeatedly until the full list is sorted.

The Algorithm for Quick sort is:-

1. Choose a pivot element from the list. This is typically the first element, but it can also be a random element or the median of the list.
2. Divide the list into two sublists: one containing elements less than the pivot and one containing elements greater than the pivot.
3. Recursively sort the sublist containing elements less than the pivot using the same process.
4. Use the same procedure to recursively sort the sublist of entries larger than the pivot.

5. Concatenate the sorted sublists with the pivot element in between to form a fully sorted list.
6. Return the fully sorted list.

(v) Merge sort: The divide-and-conquer sort algorithm divides the list into two halves, sorts each half, and then merges the two halves in sorted order.

Merge-sort Algorithm:

1. Make two sublists out of the list: one with elements below the pivot and one with elements above the pivot.
2. Produces a new sorted sublist by iteratively merging sublists until only one sublist exists. This will be your sorted list.
3. Steps to merge two sub-directories:-
4. Create an empty list to hold the sorted elements.
5. Compares the first element of each sublist.
6. Adds the smaller element to the new list and removes it from the parent sublist.
7. Repeat the steps 2 and 3 until a list is completely empty.
8. Adds the remaining elements from other sublists to a new list.
9. Replaces the merged sublist with the new sorted list.
10. Repeat this process until all sublists are merged into one sorted list.

(vi) Heapsort: A sorting algorithm that sorts elements using a data structure called heap.

This is the classification algorithm:

1. **Build max heap:** Starting with the first non-leaf node, compare each node with its child nodes and replace the nodes with the largest of its children to satisfy the max heap property.
2. **Swap root with last element:** Swap the root (largest element) with the last element in the stack.
3. Stack the rest of the elements. Starting from the root, each node is compared with its children, swapping nodes with their older children until the max heap property is satisfied.

4. Repeat steps 2 and 3 with the newly stacked elements, except for the last element in the correct position.
5. Repeat this process until only one element remains in the stack. This is now sorted.
6. **Heapify Down:** Starting from the root node, it compares elements with its children and swaps with the larger of the two until the max heap property is satisfied.
7. **Heapify Up:** Start with the last element in the heap, compare it to its parent, and swap it with the parent to satisfy the max heap property.

(vii) Radix sort: A sorting algorithm that sorts elements based on the digits or digits of their binary representation.

The Algorithm for Radix sort is:-

1. determine how many digits are contained in the input listing's largest element.
2. Initialize a variable, say digit place, to 1, which represents the current digit place.
3. Create an empty list for each possible digit value from 0 to 9.
4. Iterate through the input list and add each element to the appropriate list based on the value of the current digit place.
5. Concatenate all the lists together to form the new list in the order of the digit lists.
6. Multiply digitPlace by 10 to move to the next digit place.
7. Repeat steps 4-6 for each digit place until all digits in the largest element have been considered.
8. The final list will be sorted in ascending order by the digits of the elements.
9. Return the final sorted list.

2. Searching algorithms

C also provides the tools necessary to implement a variety of searching algorithms, such as linear search and binary search. These algorithms can quickly find specific items in a data set, making them useful for a wide range of applications.

There are many types of search algorithms.

They are:-

(i) Linear search: A basic search algorithm that examines each item in the listing one by one until it finds the desired item.

Algorithm for Linear search:-

1. Define the input for the algorithm: The input for a linear search algorithm is a list of elements (or an array) and a target element we are searching for.
2. Initialize a variable called "index" to -1: This variable will be used to store the index of the target element if it is found.
3. Loop through the list of elements: Starting from the first element, check each element in the list one by one.
4. Compare the present element to the desired element for evaluation: Keep the index of the current element in the index variable and exit the loop if the current element and the goal element are identical.
5. Return the index of the target element: After the loop completes, return the value stored in the index variable. If the target element is not found, the value of the index will be -1.

(ii) Binary search: A search algorithm that operates by splitting the listing into halves and searches within of those halves is more likely to include the element.

Algorithm for Binary search:-

1. Input: A sorted list of n elements and a target element x .
2. Initialize variables: Set the low index to 0, the high index to $n-1$, and mid to $(\text{low} + \text{high}) / 2$.
3. Start a loop: While the low index is less than or equal to the high index, repeat the following steps.
4. Compare the mid element with x : If the mid element is equal to x , return the mid index.
5. Update the low or high index: If x is greater than the mid element, set the low index to $\text{mid} + 1$. Else, set the high index to $\text{mid} - 1$.
6. Update the mid index: $\text{Mid} = (\text{low} + \text{high}) / 2$.
7. End of the loop: If the low index is greater than the high index, then x is not in the list, and the algorithm returns a failure.

8. Output: The index of x in the list or failure message.

(iii) Depth-first search: A search algorithm that examines each branch as far as is feasible before turning around.

The following guidelines apply to depth-first search:

1. select the graph's starting vertex or node to start with.
2. Add a visiting mark to the first vertex.
3. Directly place the beginning vertex into a stack.
4. Until the stack is empty, repeat the following actions: -
 - Remove the stack's top vertex.
 - Mark as visited and insert into the stack each unvisited neighbour of the popped vertex.
5. Continue this process until all vertices in the graph have been visited.
6. Once all vertices have been visited, the algorithm is complete, and a depth-first search is performed on the graph.

(iv) Breadth-first search: A search algorithm that explores all the neighbors of a node before moving to the next level.

The algorithm for the breadth-first search is:-

1. Start with the root node or the initial state.
2. Add the root node to a queue.
3. Check if the queue is empty; if yes, then terminate the algorithm.
4. Take the first element from the queue and mark it as visited.
5. Amplify the contemporary node by adding all its unvisited neighbors to the queue.
6. Until the desired node is located or the queue is empty, repeat steps 3 to 5.
7. Return the path from the preliminary state to the target state if the goal node is found.
8. Terminate the set of rules and report that the goal state was not identified if the queue is empty.

(v) Interpolation Search: A search algorithm that uses the values of the searched elements to estimate the position in the index.

It is important that the array is evenly distributed. Otherwise, it is an algorithm.

It works as expected.

The algorithm can be summarized as follows.

1. Get the input list and key value to search.
2. Initialize the lower and upper variables at the first and last indices of the list.
3. If the Lower value is less than or equal to higher value, then :-
 1. Calculate the estimated location using the following formula:
$$\text{pos} = \text{low} + ((\text{high} - \text{low}) / (\text{arr}[\text{high}] - \text{arr}[\text{low}])) * (\text{x} - \text{arr}[\text{low}]).$$
 2. Return the position if the estimated position value is a key value.
 3. c) If the estimated position value is less than the key value, set it lower.
Position + 1.
 4. d) If the value of the estimated position is greater than the key Set value, position
- 1 up.
4. If the key value is not found, return -1 to indicate that the value is not in the list.

(vi) Jump search: A search method that iterates over the list in constant-length steps until it finds the relevant element or determines that it is no longer present.

The jump search algorithm is as follows:

1. First, set the jump size to the square root of the number of array elements.
2. Sets a variable named "current" to the first element of the array.
3. Iterates over the array by jumping by jump size while incrementing a variable called "jump".
4. Move on to the following leap if the existing element is smaller than the desired element.
5. If the current element is larger than the target element, perform a linear search between the current element and the previous jump element to find the target element.

6. If the target element is not found in the array, it returns -1 to indicate that it is not in the array.
7. If the element is found, it returns the element's index in the array.

3. Graph algorithms

C's support for pointers and data structures such as arrays and linked lists makes it suitable for implementing algorithms that manipulate graphs, such as finding the shortest path between two nodes in a graph.

There are different types of graph algorithms.

they are:-

1. **Dijkstra's Algorithm:** An algorithm that finds the shortest path between two nodes in a graph by continuously updating the shortest distance from each node.
2. **Algorithm A*:** A method that continually updates the shortest course to each node in a graph to determine the shortest route between them.
3. **Prim's Algorithm:** An approach for figuring out the weighted connected graph's smallest spanning tree.
4. **Kruskal's algorithm:** An approach to identify the linked weighted graph's lowest spanning tree.
5. **Bellman-Ford Algorithm:** An algorithm that, even when the graph has negative edge weights, displays the shortest path between a particular supply node and every other node in the network.

4. Cryptographic Algorithms

C supports low-level operations and efficient data manipulation, making it ideal for implementing algorithms used in cryptography, such as data encryption and decryption algorithms.

There are different types of encryption algorithms.

They are:-

1. **Hash Algorithms:** These algorithms produce fixed-size outputs (hash) from arbitrary-sized inputs. Examples include MD5, SHA-1 and SHA-2.
2. **Symmetric key algorithms:** The encryption and decryption steps in such algorithms employ the same private key. AES, DES, and Blowfish are a few examples.
3. **Asymmetric key algorithms:** A public key and a non-public key are used by those methods as separate keys for encryption and decryption. Some examples include RSA, ECC, and DSA.
4. **Key exchange algorithms:** These algorithms allow two parties to exchange keys over an insecure channel securely. For example, we can mention Diffie-Hellman and Elliptic Curve Diffie-Hellman.

Advantages of the algorithm

they are:-

1. **Speed and efficiency:** Algorithms can process large amounts of data quickly and accurately, making them useful for tasks that are too time-consuming or error-prone for people to perform.
2. **Consistency:** Algorithms follow a set of predetermined guidelines. It can produce consistent results without being influenced by personal biases and emotions.
3. **Automation:** Algorithms can perform tasks automatically, leaving people free to focus on more complex or creative tasks.
4. **Increased accuracy:** Algorithms can often achieve higher levels of accuracy than humans, especially when dealing with large amounts of data.
5. **Better Decision Making:** Algorithms help us make more informed and objective decisions by analyzing data and identifying patterns and trends that are not easily visible to people.
6. **Scalability:** Algorithms can be easily scaled up or down to meet changing demands and workloads.

Disadvantages of the algorithm

they are:-

1. **Limited scope:** Algorithms can only solve problems within their scope and may not be able to solve complex or abstract problems.
2. **Bias:** Algorithms can perpetuate and reinforce biases in the data used for training, leading to unfair results.
3. **Insufficient transparency:** Many algorithms conceal the process through which they arrive at their conclusions. This could make it tough to think about or check the results.
4. **Reliance on the fineness of the data:** The correctness of the set of rules is heavily dependent on the fineness and applicability of the data utilised in instruction. Inaccurate or inaccurate effects may be the result of faulty data.
5. **restrained adaptability:** Algorithms are designed to follow guidelines and won't adapt to changing circumstances and conditions.

Steps of developing an algorithm

Step 1: Obtain a description of the problem. the word *client* refers to someone who wants to find a solution to a problem, and the word *developer* refers to someone who finds a way to solve the problem. The developer must create an algorithm that will solve the client's problem.

The client is responsible for creating a description of the problem, but this is often the weakest part of the process. It's quite common for a problem description to suffer from one or more of the following types of defects:

- (1) the description relies on unstated assumptions,
- (2) the description is ambiguous,
- (3) the description is incomplete, or
- (4) the description has internal contradictions.

These defects are seldom due to carelessness by the client. Instead, they are due to the fact that natural languages (English, French, Korean, etc.) are rather imprecise.

Part of the developer's responsibility is to identify defects in the description of a problem, and to work with the client to remedy those defects.

Step 2: Analyze the problem. The purpose of this step is to determine both the starting and ending points for solving the problem. This process is analogous to a mathematician

determining what is given and what must be proven. A good problem description makes it easier to perform this step.

Step 3: Develop a high-level algorithm. It's usually better to start with a high-level algorithm that includes the major part of a solution, but leaves the details until later. We can use an everyday example to demonstrate a high-level algorithm.

Step 4: Refine the algorithm by adding more detail. A high-level algorithm shows the major steps that need to be followed to solve a problem. Now we need to add details to these steps

Step 5: Review the algorithm. The final step is to review the algorithm. What are we looking for? First, we need to work through the algorithm step by step to determine whether or not it will solve the original problem.

Basic writing of algorithms

Example: - The sum of two integers.

Step 1 - Get started

Step 2 - Declare three integers a, b, c

Step 3 - Define the values of a and b

Step 4 - Add the values of a and b

Step 5 - Save the output of step 4 in c

Step 6 - Print c

Step 7 – Stop

Examples

1: Write pseudo code that reads two numbers and multiplies them together and print out their product.

2: Write pseudo code that tells a user that the number they entered is not a 5 or a 6.

- 3: Write pseudo code that performs the following: Ask a user to enter a number. If the number is between 0 and 10, write the word blue. If the number is between 10 and 20, write the word red. If the number is between 20 and 30, write the word green. If it is any other number, write that it is not a correct color option.
- 4: Write pseudo code to print all multiples of 5 between 1 and 100 (including both 1 and 100).
- 5: Write pseudo code that will count all the even numbers up to a user defined stopping point.
- 6: Write pseudo code that will perform the following.
- a) Read in 5 separate numbers.
 - b) Calculate the average of the five numbers.
 - c) Find the smallest (minimum) and largest (maximum) of the five entered numbers.
 - d) Write out the results found from steps b and c with a message describing what they are.

Homework

- 1: Write pseudo code that reads in three numbers and writes them all in sorted order.
- 2: Write pseudo code that will calculate a running sum. A user will enter numbers that will be added to the sum and when a negative number is encountered, stop adding numbers and write out the final result.

1.3.Flowchart is accurately developed based on the logic of question

Development of a flowchart

Flowchart: It is a diagram of the sequence of movements or actions of people or things involved in a complex system or activity.

It is a graphical representation of a computer program in relation to its sequence of functions (as distinct from the data it processes).

When to Use a Flowchart

- To develop understanding of how a process is done

- To study a process for improvement
- To communicate to others how a process is done
- When better communication is needed between people involved with the same process
- To document a process
- When planning a project

Flowchart Basic Procedure

Materials needed: Sticky notes or cards, a large piece of flipchart paper or newsprint, and marking pens.

1. Define the process to be diagrammed. Write its title at the top of the work surface.
2. Discuss and decide on the boundaries of your process: Where or when does the process start? Where or when does it end? Discuss and decide on the level of detail to be included in the diagram.
3. Brainstorm the activities that take place. Write each on a card or sticky note.
4. Arrange the activities in proper sequence.
5. When all activities are included and everyone agrees that the sequence is correct, draw arrows to show the flow of the process.
6. Review the flowchart with others involved in the process (workers, supervisors, suppliers, customers) to see if they agree that the process is drawn accurately.

Here is a chart for some of the common symbols used in drawing flowcharts.

Symbol	Symbol Name	Purpose
	Start/Stop	Used at the beginning and end of the algorithm to show start and end of the program.
	Process	Indicates processes like mathematical operations.
	Input/ Output	Used for denoting program inputs and outputs.

	Decision	Stands for decision statements in a program, where answer is usually Yes or No.
	Arrow	Shows relationships between different shapes.
	On-page Connector	Connects two or more parts of a flowchart, which are on the same page.
	Off-page Connector	Connects two parts of a flowchart which are spread over different pages.

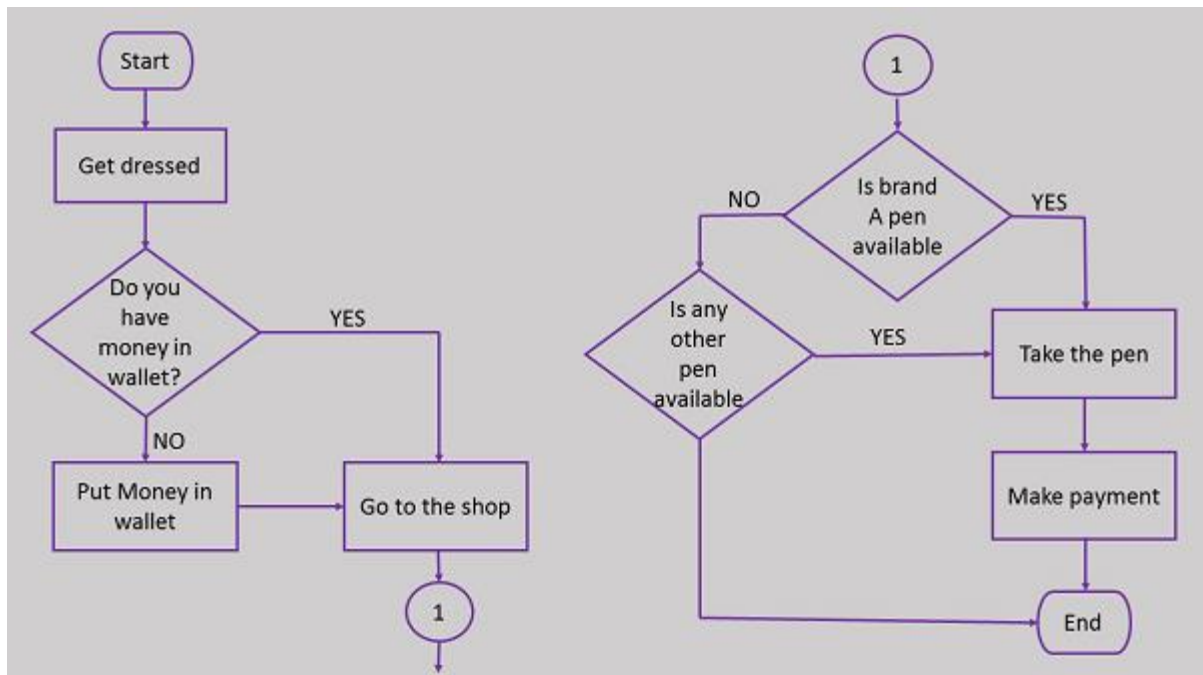
Guidelines for Developing Flowcharts

These are some points to keep in mind while developing a flowchart –

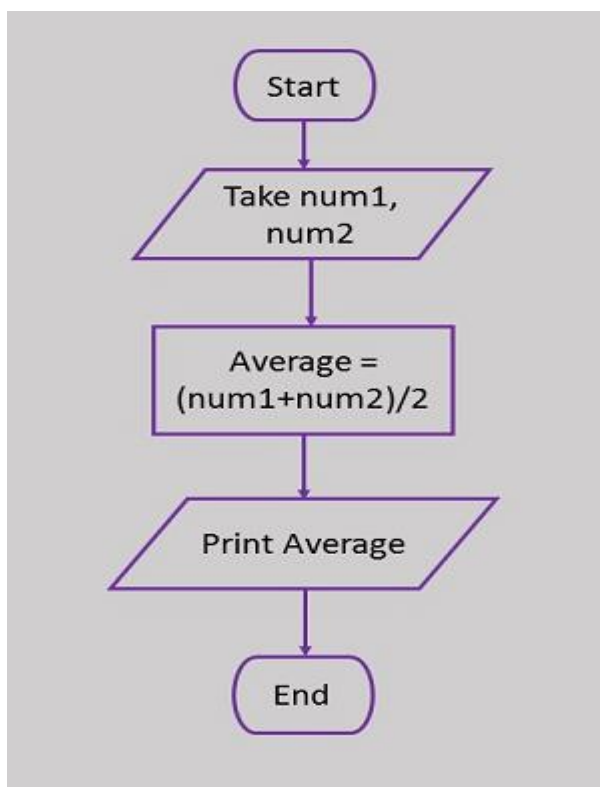
- Flowchart can have only one start and one stop symbol
- On-page connectors are referenced using numbers
- Off-page connectors are referenced using alphabets
- General flow of processes is top to bottom or left to right
- Arrows should not cross each other

Example Flowcharts

Here is the flowchart for going to the market to purchase a pen.



Here is a flowchart to calculate the average of two numbers.



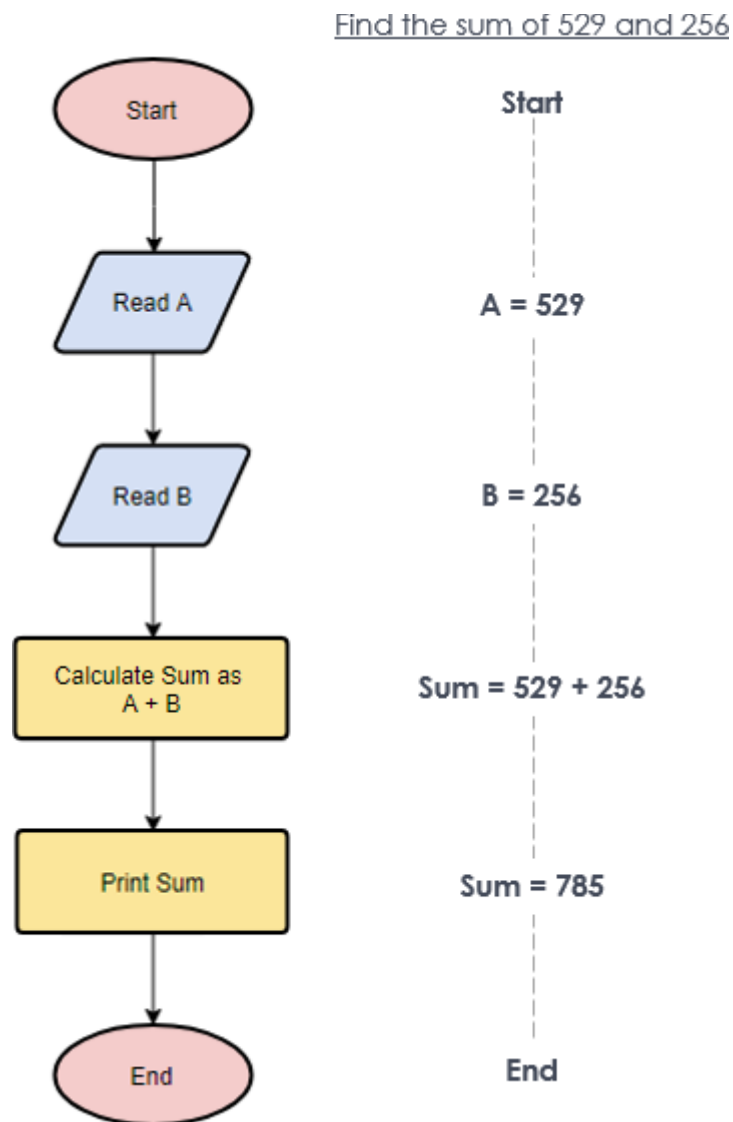
https://www.tutorialspoint.com/programming_methodologies/programming_methodologies_flowchart_elements.htm

<https://asq.org/quality-resources/flowchart>

Examples of flowchart

Flowchart Example – Simple Algorithms

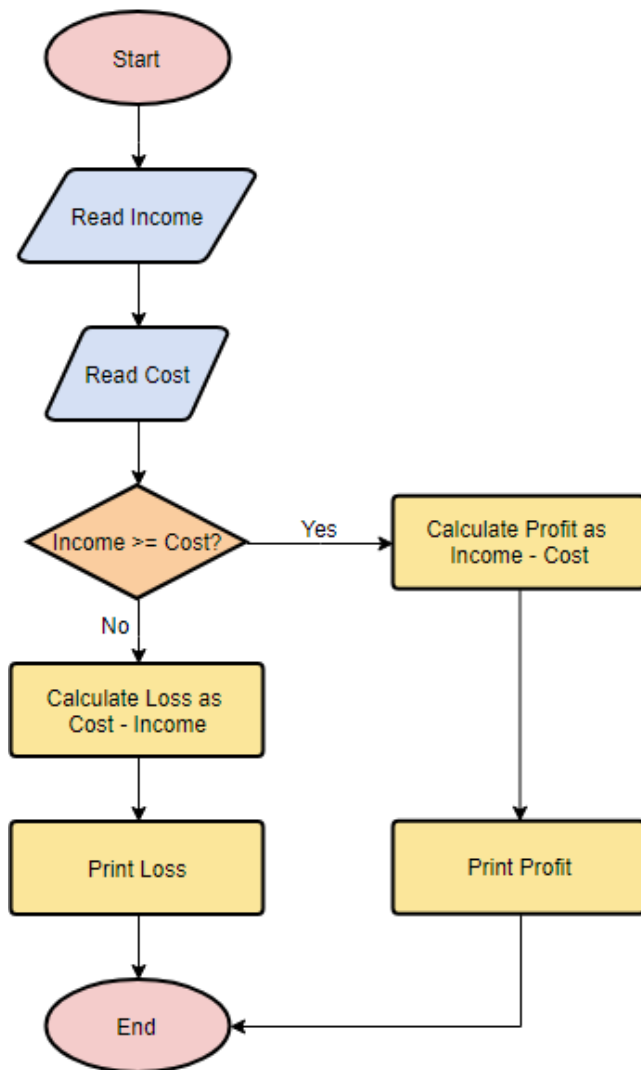
A flowchart can also be used in visualizing algorithms, regardless of its complexity. Here is an example that shows how flowchart can be used in showing a simple summation process.



Flowchart Example – Calculate Profit and Loss

The flowchart example below shows how profit and loss can be calculated.

Find the profit/loss when
income = 1,000, cost = 800



<https://www.visual-paradigm.com/tutorials/flowchart-tutorial/>

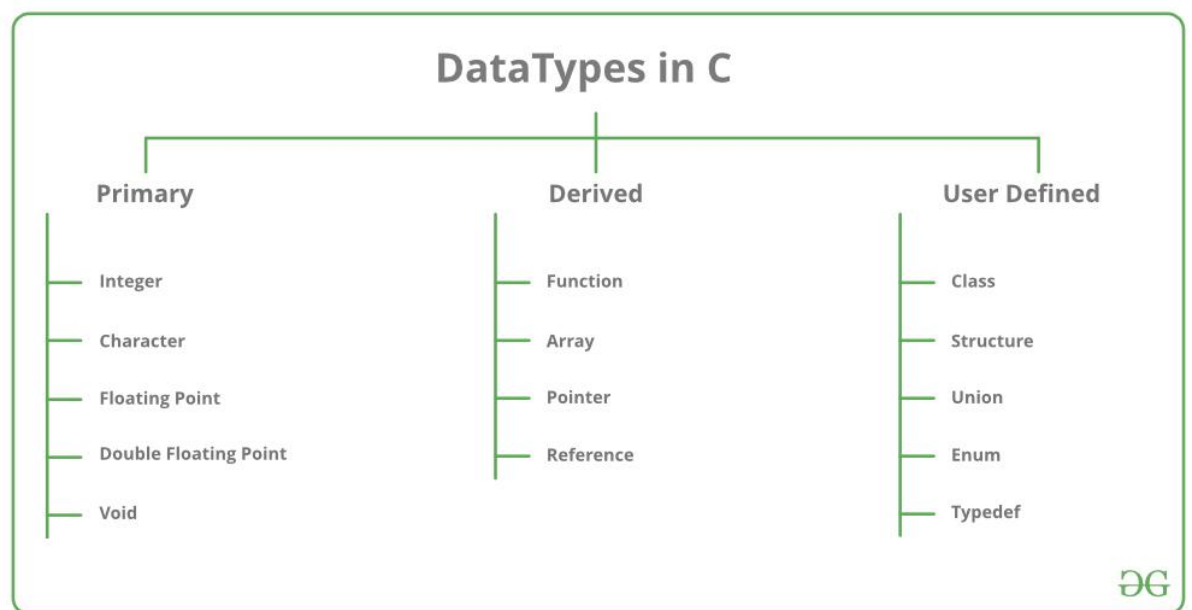
Learning outcome TWO: Write C Programming codes

Description of C programming building blocks

- **Tokens in C:** A token is the smallest unit used in a C program. Each and every punctuation and word that you come across in a C program is token. A compiler breaks a C program into tokens and then proceeds ahead to the next stages used in the compilation process.
- **Datatypes:** It specifies the type of data that the variable can store like integer, character, floating, double, etc.

The data types in C can be classified as follows:

Types	Description
Primitive Data Types	Primitive data types are the most basic data types that are used for representing simple values such as integers, float, characters, etc.
User Defined Data Types	The user-defined data types are defined by the user himself.
Derived Types	The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types.



Data Type	Size (bytes)	Format Specifier
short int	2	%hd
unsigned short int	2	%hu
unsigned int	4	%u
int	4	%d

Data Type	Size (bytes)	Format Specifier
long int	4	%ld
unsigned long int	4	%lu
long long int	8	%lld
unsigned long long int	8	%llu
signed char	1	%c
unsigned char	1	%c
float	4	%f
double	8	%lf
long double	16	%Lf

The following are some main primitive data types in C:

Integer Data Type

The integer datatype in C is used to store the integer numbers(any number including positive, negative and zero without decimal part). Octal values, hexadecimal values, and decimal values can be stored in int data type in C.

Character Data Type

Character data type allows its variable to store only a single character. The size of the character is 1 byte. It is the most basic data type in C. It stores a single character and requires a single byte of memory in almost all compilers.

Float Data Type

In C programming float data type is used to store floating-point values. Float in C is used to store decimal and exponential values. It is used to store decimal numbers (numbers with floating point values) with single precision.

Double Data Type

A Double data type in C is used to store decimal numbers (numbers with floating point values) with double precision. It is used to define numeric values which hold numbers with decimal values in C.

Void Data Type

The void data type in C is used to specify that no value is present. It does not provide a result value to its caller. It has no values and no operations. It is used to represent nothing. Void is used in multiple ways as function return type, function arguments as void, and pointers to void.

Size of Data Types in C

The size of the data types in C is dependent on the size of the architecture, so we cannot define the universal size of the data types. For that, the C language provides the `sizeof()` operator to check the size of the data types.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int size_of_int = sizeof(int);
```

```
    int size_of_char = sizeof(char);
```

```
    int size_of_float = sizeof(float);
```

```

int size_of_double = sizeof(double);

printf("The size of int data type : %d\n", size_of_int);
printf("The size of char data type : %d\n",
    size_of_char);
printf("The size of float data type : %d\n",
    size_of_float);
printf("The size of double data type : %d",
    size_of_double);

return 0;
}

```

Output

The size of int data type : 4

The size of char data type : 1

The size of float data type : 4

The size of double data type : 8

<https://www.geeksforgeeks.org/data-types-in-c/>

- **Variables:** In C programming language, a variable is a user-defined or a user-readable custom name assigned to a memory location. Variables hold a value that can be modified and reused many times during the program execution.

A variable can be an alphabet or digits and start with an underscore, but you cannot declare a keyword as a variable name.

Declaration of Variables in C

A few simple parameters are considered to declare a variable in the C programming language. These parameters are collectively termed syntax.

The following syntax is strictly followed to declare any variable in C:

Syntax:

```
<data type> <variable-list>;
```


Example:

```
int a, b, sum;
```

Initialization of Variables in C

There are two ways to utilize variables in C. To utilize variables, you either have to declare them or initialize them at the declaration stage.

The following will explain the process in an elaborate yet simple manner.

Syntax:

```
<data type> <variable-name = value>;
```

You may also declare and initialize multiple variables at the same time as given below:

Syntax:

```
<data type> <var1 = val1>, <var2 = val2>,.....<varn = valn>;
```

Example:

```
int a = 10, b=20,c;
```

You have come across the technical definition, declaration, and initialization of Variables in C. Now, you have an important stage. There are rules and regulations to declare and initialize the variables in C.

Now, you will go through the rules and regulations to have a smoother learning experience.

- The identifiers must not begin with a numerical digit.
- The first character used in an identifier should be either an underscore or an alphabet. After that, any of the characters, underscores, or digits can follow it.
- Both- the lowercase and uppercase letters are distinct in an identifier. Thus, we can safely say that an identifier is case-sensitive.
- We cannot use an identifier for representing the keywords.
- An identifier does not specify blank spaces or commas.

- The maximum length of an identifier is 31 characters.
- We must write identifiers in such a way that it is not only meaningful- but also easy to read and short.

Keywords in C

We can define the keywords as the reserved or pre-defined words that hold their own importance. It means that every keyword has a functionality of its own.

The C language provides a support for 32 keywords, as mentioned below:

auto	enum	const	goto
double	case	float	default
struct	register	unsigned	sizeof
int	typedef	short	volatile
break	extern	continue	if
else	char	for	do
switch	return	void	static
long	union	signed	while

<https://www.simplilearn.com/tutorials/c-tutorial/what-is-variables-in-c>

- **Constant:** A constant is very similar to variables in the C programming language, but it can hold only a single variable during the execution of a program. It means that once we assign value to the constant, then we can't change it throughout the execution of a program- it stays fixed.

[https://byjus.com/gate/constants-in-](https://byjus.com/gate/constants-in-c/#:~:text=A%20constant%20is%20very%20similar,a%20program%2D%20it%20stays%20fixed.)

[c/#:~:text=A%20constant%20is%20very%20similar,a%20program%2D%20it%20stays%20fixed.](https://byjus.com/gate/constants-in-c/#:~:text=A%20constant%20is%20very%20similar,a%20program%2D%20it%20stays%20fixed.)

- **Operators:** The operators are types of symbols that inform a compiler for performing some specific logical or mathematical functions. The operators serve as the foundations of the programming languages. Thus, the overall functionalities of the C programming language remain incomplete if we do not use operators.

Types of Operators in C

Various types of operators are available in the C language that helps a programmer in performing different types of operations. We can handle different operations in a program using these types of operators:

- Relational Operators
- Arithmetic Operators
- Logical Operators
- Assignment Operators
- Bitwise Operators
- Misc Operators

Relational Operators

We use the relational operators in c when we would like to compare the values available for two given operands. Here is a table that states all the relational operators available in the C language, along with their individual functions.

Function of Operator	Operator	Example
To check if two operands are equal to each other.	==	5 == 3 would return to 0 and 4 == 4 would return to 1
To check if two operands are unequal to each other.	!=	8 != 5 would return to 1 6 != 6 would return to 0
To check if the operand value on the left is comparatively greater than that of the operand value on the right.	>	5 > 8 would return to 0 9 > 7 would return to 1

To check if the operand value on the left is comparatively smaller than that of the operand value on the right.	<	2 < 0 would return to 0 4 < 8 would return to 1
To check if the operand value on the left is equal to or comparatively greater than that of the operand value on the right.	>=	1 >= 5 would return to be 0 4 >= 2 would return to be 1 9 >= 9 would return to be 1
To check if the operand value on the left is equal to or comparatively smaller than that of the operand value on the right.	<=	3 <= 6 would return to be 1 7 <= 3 would return to be 0 2 <= 2 would return to be 1

Arithmetic Operators

The arithmetic operators in C language help a user perform the mathematical operations as well as the arithmetic operations in a program, such as subtraction (-), addition (+), division (/), multiplication (*), the remainder of division (%), decrement (--), increment (++).

The arithmetic operators are of two major types:

- **Binary Operators** – It works using two of the operators, such as -, +, /, *.
- **Unary Operators In C** – It works by making use of just a single operand (value), such as — and ++.
- **Binary Operators**
- Here is a table that states all the binary arithmetic operators available in the C language, along with their individual functions. If P = 50 and Q = 25, then:

Function of Operator	Operator	Example
To subtract the value of the second operator from the first one.	-	$P - Q = 25$
To add the value of the second operator with the first one.	+	$P + Q = 75$
To divide the value of the numerator with the value of the denominator.	/	$P / Q = 2$
To multiply the value of the second operator with the first one.	*	$P * Q = 1250$

Unary Operators

Here is a table that states all the unary arithmetic operators available in the C language, along with their individual functions. These are increment and decrement operators. If $P = 50$ and $Q = 25$, then:

Function of Operator	Operator	Example
To increase the value of the integer/ operator by 1.	Decrement Operator	$P -- = 49$ and $Q -- = 24$
To decrease the value of the integer/ operator by 1.	Increment Operator	$P ++ = 51$ and $Q ++ = 26$

Prefix and Postfix Increment/ Decrement Operators

The decrement operators and increment operators are of two major types:

Prefix – When we use the operator before the available variable in a program as a prefix, it is known as a prefix operator. In a prefix increment operator, the program first adds 1 and then assigns the resultant value to the variable. While in a prefix decrement operator, the program first subtracts 1 and then assigns the resultant value to the variable. For example, $++a$ and $--a$.

Postfix – When we use the operator after the available variable in a program as a postfix, it is known as a postfix operator. In a postfix increment operator, the program first assigns the value to the variable, then adds 1, and then assigns the resultant value. While in a postfix

decrement operator, the program first assigns the value to the variable, then subtracts 1, and then assigns the resultant value. For example, `a++` and `a--`.

Here is a table that states all the prefix and postfix unary arithmetic operators available in the C language, along with their individual functions. These are increment and decrement operators.

Type of Operator	Sample Operator Expression	Description/ Explanation
Prefix Increment Operator	<code>++p</code>	p increases by a value of 1, then the program uses the value of p.
Postfix Increment Operator	<code>p++</code>	The program uses the current value of p and increases the value of p by 1.
Prefix Decrement Operator	<code>--p</code>	p decreases by a value of 1, then the program uses the value of p.
Postfix Decrement Operator	<code>p--</code>	The program uses the current value of p and decreases the value of p by 1.

Logical Operators

The logical operators in c help a user determine if the results would be true or false. Here is a table that states all the logical operators available in the C language, along with their individual functions. If `P = 1` and `Q = 0`, then:

Name of Operator	Operator	Description of Operator	Example
Logical NOT	<code>!</code>	We use this operator for reversing the available logical state of the operand. In case the condition turns out to be true, then this operator will make the value false.	<code>!(P && Q)</code> turns out to be true.
Logical OR	<code> </code>	The given condition turns out to be true if any of the operands happen to be non-zero.	<code>(P Q)</code> turns out to be true.

Logical AND	&&	The given condition turns out to be true if only both the operands happen to be non-zero.	(P && Q) turns out to be false.
----------------	----	---	---------------------------------

Assignment Operators

We use the assignment operators in c for assigning any value to the given variable. Here is a table that states all the logical operators available in the C language, along with their individual functions.

Function of Operator	Operator	Example
To assign the values from the right operand to the left operand.	=	p = q
To add the value of the right operand with the value of the left operand. After that, assign the resultant value to the left one.	+=	p += q is similar to that of p = p+q
To subtract the value of the right operand from the value of the left operand. After that, assign the resultant value to the left one.	-=	p -= q is similar to that of p = p-q
To multiply the value of the right operand with the value of the left operand. After that, assign the resultant value to the left one.	*=	p *= q is similar to that of p = p*q
To divide the value of the right operand with the value of the left operand. After that, assign the resultant value to the left one.	/=	p /= q is similar to that of p = p/q
To calculate the modulus of the values of the right operand and the left operand. After that, assign the resultant value to the left one.	%=	p %= q is similar to that of p = p%q

Bitwise Operators

We use bitwise operators in c for performing the operations of bit-level on various operands. It first converts the operators to bit-level, and after that, it performs various calculations.

The Bitwise operators basically work on the bits, and we perform bit-by-bit operations using them. Here is the truth table for the $\wedge$, $\&$, and $|$ here:

a	b	a & b	a b	a ^ b
1	1	1	1	0
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1

If we assume that P = 60 and Q = 13 using the binary format, then it will appear as follows:

P = 0011 1100

Q = 0000 1101

P ^ Q = 0011 0001

P | Q = 0011 1101

P & Q = 0000 1100

~ P = 1100 0011

Here is a table that states all the bitwise operators available in the C language, along with their individual functions. If we assume that the value of variable P = 60 and variable Q = 13, then:

Name of Operator	Description of Operator	Operator	Example
------------------	-------------------------	----------	---------

Binary AND	&	This operator copies the bits to the result only if they exist in both of the operands.	(P & Q) = 12, The binary value is 0000 1100
Binary OR		This operator copies the bits only if they exist in either of the operands.	(P Q) = 61, The binary value is 0011 1101
Binary XOR	^	This operator copies the bits only if they are set in one of the operands but not in both of them.	(P ^ Q) = 49, The binary value is 0011 0001
Binary One's Complement	~	This operator is unary. Also, it performs the effect of 'flipping' the available bits.	(~P) = ~(60), The binary value is - 0111101
Binary Left Shift	<<	It shifts the value of the left operand to the left on the basis of the number of bits that the right operand specifies.	P << 2 = 240, The binary value is 1111 0000
Binary Right Shift	>>	It shifts the value of the left operand to the right on the basis of the number of bits that the right operand specifies.	P >> 2 = 15, The binary value is 0000 1111

Misc Operators

Apart from all the operators that we have discussed above, the C programming language uses a few other crucial operators, that include operators, (such as ? : and sizeof). Here is a table

that states all the misc operators available in the C language, along with their individual functions.

Description of Operator	Operator	Example
To return the actual size of any given variable.	Size of()	If z is an integer, then the size of(z) will return to be 4.
To return the address of any given variable.	&	For instance, &z; would return the actual address for any given variable.
Pointer to any variable.	*	*z;
Conditional Expression.	? :	When the Condition is true ? then the value X : In any other case, the value Y

<https://byjus.com/gate/operators-in-c/>

- **Description of C program structure**

A C program basically consists of the following parts –

- Preprocessor Commands
- Functions
- Variables
- Statements & Expressions
- Comments

Let us look at a simple code that would print the words "Hello World" –

```
#include <stdio.h>
int main() {
    /* my first program in C */
    printf("Hello, World! \n");
    return 0;
}
```

Let us take a look at the various parts of the above program –

- The first line of the program `#include <stdio.h>` is a preprocessor command, which tells a C compiler to include `stdio.h` file before going to actual compilation.
- The next line `int main()` is the main function where the program execution begins.
- The next line `/*...*/` will be ignored by the compiler and it has been put to add additional comments in the program. So such lines are called comments in the program.
- The next line `printf(...)` is another function available in C which causes the message "Hello, World!" to be displayed on the screen.
- The next line **`return 0;`** terminates the `main()` function and returns the value 0.

Preprocessor Commands: The **C Preprocessor** is not a part of the compiler, but is a separate step in the compilation process. In simple terms, a C Preprocessor is just a text substitution tool and it instructs the compiler to do required pre-processing before the actual compilation.

All preprocessor commands begin with a hash symbol (#). It must be the first nonblank character, and for readability, a preprocessor directive should begin in the first column. The following section lists down all the important preprocessor directives –

Sr.No.	Directive & Description
1	#define Substitutes a preprocessor macro.
2	#include Inserts a particular header from another file.
3	#undef Undefines a preprocessor macro.
4	#ifdef Returns true if this macro is defined.
5	#ifndef Returns true if this macro is not defined.

6	#if Tests if a compile time condition is true.
7	#else The alternative for #if.
8	#elif #else and #if in one statement.
9	#endif Ends preprocessor conditional.
10	#error Prints error message on stderr.
11	#pragma Issues special commands to the compiler, using a standardized method.

- A **function** is a group of statements that together perform a task. Every C program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.

Predefined Functions

A pre-defined function is built into the software and does not need to be created by a programmer. Pre-defined functions often exist to carry out common tasks, such as:

- finding an average number
- determining the length of a string

For example, **main()** is a function, which is used to execute code, and **printf()** is a function; used to output/print text to the screen:

Example

```
int main() {
    printf("Hello World!");
    return 0;
}
```

- **Expression and statements**

An *expression* represents a single data item--usually a number. The expression may consist of a single entity, such as a constant or variable, or it may consist of some combination of such entities, interconnected by one or more *operators*. Expressions can also represent logical conditions which are either true or false. However, in C, the conditions true and false are represented by the integer values 1 and 0, respectively. Several simple expressions are given below:

- $a + b$ employs the *addition operator* (+), represents the sum of the values assigned to variables a and b.
- $x = y$ involves the *assignment operator* (=), and causes the value represented by y to be assigned to x.
- $t = u + v$ the value of the expression $(u + v)$ is assigned to t.
- $x \leq y$ takes the value 1 (true) if the value of x is less than or equal to the value of y. $\leq$ is a *relational operator* that compares the values of x and y.
- $++j$ causes the value of j to be increased by 1. Thus, the expression is equivalent to $j = j + 1$

A *statement* causes the computer to carry out some definite action. There are three different classes of statements in C: *expression statements*, *compound statements*, and *control statements*.

- An expression statement consists of an expression followed by a semicolon. The execution of such a statement causes the associated expression to be evaluated. For example:

```
a = 6;
```

```
c = a + b;
```

```
++j;
```

- A compound statement consists of several individual statements enclosed within a pair of braces { }. A typical compound statement is shown below:

```
{  
pi = 3.141593;  
circumference = 2. * pi * radius;  
area = pi * radius * radius;  
}
```

- A *symbolic constant* is a name that substitutes for a sequence of characters. Symbolic constants are usually defined at the beginning of a program, by writing

```
#define NAME text
```

For instance, suppose that a C program contains the following symbolic constant definition:

```
#define PI 3.141593
```

Suppose, further, that the program contains the statement

```
area = PI * radius * radius;
```

During the compilation process, the preprocessor replaces each occurrence of the symbolic constant PI by its corresponding text. Hence, the above statement becomes

```
area = 3.141593 * radius * radius;
```

Variables: Variables are the names you give to computer memory locations which are used to store values in a computer program.

Creating variables

Creating variables is also called **declaring variables** in C programming.

```
#include <stdio.h>  
  
int main() {  
    int a;  
    int b;  
}
```

You can create variables of similar type by putting them in a single line but separated by comma as follows –

```
#include <stdio.h>
```

```
int main() {  
    int a, b;  
}
```

Store Values in Variables

You have seen how we created variables in the previous section. Now, let's store some values in those variables –

```
#include <stdio.h>  
int main() {  
    int a;  
    int b;  
    a = 10;  
    b = 20;  
}
```

Access stored values in variables

We know that the above program has two variables **a** and **b** and they store the values 10 and 20, respectively. So let's try to print the values stored in these two variables. Following is a C program, which prints the values stored in its variables –

```
#include <stdio.h>  
int main() {  
    int a;  
    int b;  
    a = 10;  
    b = 20;  
    printf( "Value of a = %d\n", a );  
    printf( "Value of b = %d\n", b );  
}
```

When the above program is executed, it produces the following result –

```
Value of a = 10
```

Value of b = 20

You must have seen **printf()** function in the previous chapter where we had used it to print "Hello, World!". This time, we are using it to print the values of variables. We are making use of **%d**, which will be replaced with the values of the given variable in **printf()** statements. We can print both the values using a single **printf()** statement as follows –

```
#include <stdio.h>

int main() {
    int a;
    int b;
    a = 10;
    b = 20;
    printf( "Value of a = %d and value of b = %d\n", a, b );
}
```

When the above program is executed, it produces the following result –

Value of a = 10 and value of b = 20

NB: If you want to use **float** variable in C programming, then you will have to use **%f** instead of **%d**, and if you want to print a character value, then you will have to use **%c**. Similarly, different data types can be printed using different % and characters.

Comments in C

Comments can be used to explain code, and to make it more readable. It can also be used to prevent execution when testing alternative code.

Comments can be **singled-lined** or **multi-lined**.

Single-line Comments

Single-line comments start with two forward slashes (**//**).

Any text between **//** and the end of the line is ignored by the compiler (will not be executed).

This example uses a single-line comment before a line of code:

Example

```
// This is a comment  
printf("Hello World!");
```

C Multi-line Comments

Multi-line comments start with `/*` and ends with `*/`.

Any text between `/*` and `*/` will be ignored by the compiler:

Example

```
/* The code below will print the words Hello World!  
to the screen, and it is amazing */  
printf("Hello World!");
```